



МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Дальневосточный федеральный университет»

ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ
Кафедра математического и компьютерного моделирования

Избосарова Жасмина Олимжоновна

**РЕШЕНИЕ УРАВНЕНИЙ МАТЕМАТИЧЕСКОЙ ФИЗИКИ МЕТОДОМ PINN
И АНАЛИЗ УСТОЙЧИВОСТИ РЕШЕНИЯ МЕТОДОМ НЕЙРОННОГО
ТАНГЕНЦИАЛЬНОГО ЯДРА**

КУРСОВАЯ РАБОТА

Направление подготовки 02.03.01 сэт Сквозные цифровые технологии,
бакалавриатская программа «Математика и компьютерные науки»
Очной формы обучения

Студент гр. Б9123-02.03.01 сэт _____
(подпись)

Руководитель _____ К.С.Кузнецов
(подпись)

Оценка _____

Регистрационный № _____

(подпись) И.О. Фамилия

(подпись) И.О. Фамилия

«_____» _____ 2026 г.

«_____» _____ 2026 г.

г. Владивосток
2026

Введение

Среди современных методов численного решения уравнений в частных производных заметное место занимают Physics-Informed Neural Networks (PINN) [4], объединяющие нейросетевую аппроксимацию с явным учётом физических законов. Такой подход особенно важен в задачах, где объём наблюдаемых данных ограничен, а устойчивость и согласованность решения с математической моделью критичны.

В данной работе исследуется численное решение уравнения Курамото–Сивашинского методом PINN. Это уравнение описывает сложную нелинейную пространственно-временную динамику и используется, в частности, при моделировании фронта пламени и неустойчивых интерфейсов [2; 5]. Практический интерес к нему связан с жёсткостью и чувствительностью к численным ошибкам, что делает его удобным тестом для оценки возможностей PINN.

Исследование включает два этапа. На первом этапе решается исходное уравнение четвёртого порядка. На втором этапе выполняется понижение порядка за счёт введения вспомогательной переменной $v = \frac{\partial^2 u}{\partial x^2}$, после чего исходная задача сводится к системе уравнений второго порядка. Основная цель работы состоит в сравнении этих двух постановок и в анализе причин, по которым формально более простая система второго порядка показала худшую точность и менее устойчивую динамику обучения.

Глава 1 Математическая постановка задачи и метод решения

1.1 Математическая модель и ключевые свойства

В данной работе рассматривается одномерная задача для уравнения Курамото–Сивашинского, являющегося нелинейным эволюционным уравнением в частных производных. В используемой постановке уравнение имеет вид формула (1.1):

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} + \frac{\partial^2 u}{\partial x^2} + \frac{\partial^4 u}{\partial x^4} = 0, \quad x \in [-L, L], \quad t \in [0, T]. \quad (1.1)$$

Данное уравнение описывает сложную пространственно-временную динамику и включает несколько физических механизмов. Оно используется, в частности, для описания эволюции неустойчивых интерфейсов и фронтов пламени [2; 5]. Нелинейный конвективный член $u \frac{\partial u}{\partial x}$ отвечает за перенос возмущений, диффузионный член второго порядка $\frac{\partial^2 u}{\partial x^2}$ в данной записи отвечает за линейную неустойчивость, тогда как член четвёртого порядка $\frac{\partial^4 u}{\partial x^4}$ выполняет стабилизирующую функцию, подавляя рост высокочастотных возмущений.

Для рассматриваемой задачи известно точное аналитическое решение в виде бегущей волны формула (1.2):

$$u_{\text{exact}}(x, t) = b + \frac{15}{19} \sqrt{\frac{11}{19}} \left(-9 \tanh(k(x - bt - x_0)) + 11 \tanh^3(k(x - bt - x_0)) \right). \quad (1.2)$$

где параметры заданы в формула (1.3):

$$b = 5, \quad k = \frac{1}{2} \sqrt{\frac{11}{19}}, \quad x_0 = -12. \quad (1.3)$$

Данное решение получено путём редукции исходного уравнения в частных производных к обыкновенному дифференциальному уравнению с использованием замены переменной формула (1.4):

$$\xi = x - bt. \quad (1.4)$$

что соответствует переходу к системе координат, движущейся со скоростью

b , и позволяет представить решение в форме бегущей волны.

Начальные и граничные условия для рассматриваемой задачи задаются соотношениями формулы (1.5)–(1.7):

$$u(0, x) = u_0(x). \quad (1.5)$$

$$u(t, -L) = u_1(t), \quad u(t, L) = u_2(t). \quad (1.6)$$

$$\frac{\partial u}{\partial x}(t, -L) = u_3(t), \quad \frac{\partial u}{\partial x}(t, L) = u_4(t). \quad (1.7)$$

Начальное условие задаётся формула (1.5) и определяет распределение искомой функции $u(x, t)$ в начальный момент времени $t = 0$.

Граничные условия Дирихле задаются формула (1.6); они фиксируют значения функции на границах пространственной области $x = -L$ и $x = L$ для всех моментов времени.

Условия Неймана задаются формула (1.7) и определяют значения пространственной производной $\frac{\partial u}{\partial x}$ на границах области. Эти условия описывают поток или градиент величины u через границы и играют важную роль в обеспечении корректности постановки задачи, особенно для уравнений более высокого порядка.

Совместное задание начального условия и граничных условий Дирихле и Неймана формирует корректно поставленную краевую задачу для уравнения Курамото–Сивашинского и обеспечивает единственность и устойчивость её решения.

Выбор данной задачи обусловлен наличием точного решения, что делает её удобной для верификации численных методов. В частности, это позволяет количественно оценивать точность и устойчивость метода Physics-Informed Neural Networks (PINN), а также его способность воспроизводить нелинейную динамику одномерного уравнения Курамото–Сивашинского в условиях ограниченного объёма данных.

1.2 Описание метода Physics-Informed Neural Networks

Метод Physics-Informed Neural Networks (PINN) представляет собой подход, в котором нейронная сеть используется для аппроксимации

решения дифференциального уравнения с явным учётом физических законов, задаваемых этим уравнением. В базовой постановке искомая функция приближается полносвязной нейронной сетью, а обучение осуществляется посредством минимизации составной функции потерь, включающей как вклад невязки дифференциального уравнения, так и отклонения от начальных и граничных условий [4].

Реализация выполнена в среде Python с применением библиотек автоматического дифференцирования и градиентной оптимизации. Общая методология обучения не зависит от конкретного вычислительного эксперимента и включает три ключевых этапа:

- формирование точек начального условия (IC), граничных условий Дирихле и Неймана (BC), а также внутренних коллокационных точек;
- вычисление невязки уравнения и граничных ограничений посредством автоматического дифференцирования;
- минимизацию взвешенной составной функции потерь.

Искомая функция аппроксимируется полносвязной нейронной сетью прямого распространения формула (1.8):

$$u_{\theta}(t, x) \approx u(t, x), \quad (1.8)$$

где θ — вектор параметров сети.

Входными параметрами сети являются пространственная и временная координаты, подвергнутые предварительному преобразованию, а выходом — значение искомой функции. Архитектура сети подбирается таким образом, чтобы обеспечить достаточную выразительную способность для аппроксимации нелинейного решения с выраженными пространственно-временными структурами.

Обучение модели осуществляется на совокупности точек рисунок 1, распределённых в области определения задачи. Используются три типа точек: точки начального условия, точки граничных условий и внутренние коллокационные точки, в которых контролируется выполнение дифференциального уравнения.

Внутренние точки используются для минимизации невязки дифференциального уравнения, получаемой путём подстановки нейросетевой



Рисунок 1 – Обучающая выборка

аппроксимации в исходное уравнение. Производные по пространственной и временной переменным, включая старшие производные, вычисляются с использованием автоматического дифференцирования.

Невязка уравнения имеет вид формула (1.9):

$$r_{\theta}(t, x) = \frac{\partial u_{\theta}}{\partial t} + u_{\theta} \frac{\partial u_{\theta}}{\partial x} + \frac{\partial^2 u_{\theta}}{\partial x^2} + \frac{\partial^4 u_{\theta}}{\partial x^4}. \quad (1.9)$$

Функция потерь формируется как сумма нескольких слагаемых формулы (1.10)–(1.14):

$$\mathcal{L} = \lambda_r \mathcal{L}_r + \lambda_0 \mathcal{L}_0 + \lambda_D \mathcal{L}_D + \lambda_N \mathcal{L}_N, \quad (1.10)$$

где

$$\mathcal{L}_r = \frac{1}{N_r} \sum_{i=1}^{N_r} |r_{\theta}(t_i, x_i)|^2, \quad (1.11)$$

$$\mathcal{L}_0 = \frac{1}{N_0} \sum_{i=1}^{N_0} |u_{\theta}(0, x_i) - u_0(x_i)|^2, \quad (1.12)$$

$$\mathcal{L}_D = \frac{1}{N_b} \sum_{i=1}^{N_b} (|u_{\theta}(t_i, -L) - u_1(t_i)|^2 + |u_{\theta}(t_i, L) - u_2(t_i)|^2). \quad (1.13)$$

$$\mathcal{L}_N = \frac{1}{N_b} \sum_{i=1}^{N_b} \left(\left| \frac{\partial u_\theta}{\partial x}(t_i, -L) - u_3(t_i) \right|^2 + \left| \frac{\partial u_\theta}{\partial x}(t_i, L) - u_4(t_i) \right|^2 \right). \quad (1.14)$$

Здесь N_r , N_0 , N_b — количество внутренних, начальных и граничных точек соответственно, а λ_r , λ_0 , λ_D , λ_N — весовые коэффициенты.

Каждая из компонент в формула (1.10) имеет самостоятельный физико-математический смысл. Слагаемое формула (1.11) отвечает за выполнение самого дифференциального уравнения во внутренних коллокационных точках области и тем самым обеспечивает физическую согласованность нейросетевой аппроксимации. Компонента формула (1.12) контролирует точность воспроизведения начального условия и заставляет сеть корректно восстанавливать профиль решения в момент времени $t = 0$. Слагаемое формула (1.13) штрафует отклонения значений $u_\theta(t, x)$ от заданных граничных условий Дирихле на концах пространственного интервала, то есть фиксирует значения решения на границе области. Наконец, компонента формула (1.14) отвечает за выполнение граничных условий Неймана и согласует значения пространственной производной $\frac{\partial u_\theta}{\partial x}$ на границах с предписанными функциями $u_3(t)$ и $u_4(t)$. Таким образом, минимизация полной функции потерь означает одновременное стремление сети удовлетворить уравнению, начальному условию и обоим типам граничных ограничений, а весовые коэффициенты позволяют регулировать относительный вклад каждой из этих составляющих в процесс обучения.

Обучение сводится к задаче оптимизации формула (1.15):

$$\theta^* = \arg \min_{\theta} \mathcal{L}. \quad (1.15)$$

С целью повышения качества восстановления решения в работе применяется нормирование искомой функции формула (1.16):

$$u(t, x) = u_s \cdot \tilde{u}(t, x), \quad (1.16)$$

где u_s — масштабный коэффициент.

Для рассматриваемой задачи важна корректная передача высокочастотных компонент и локальных перепадов решения. Однако классические полносвязные сети проявляют эффект спектрального смещения (spectral bias), то есть склонность сначала аппроксимировать низкочастотные составляющие

[1; 3]. Для ослабления этого эффекта входные переменные дополнительно преобразуются с помощью Фурье-кодирования формула (1.17):

$$\gamma(t) = [\sin(2\pi B_t t), \cos(2\pi B_t t)], \quad \gamma(x) = [\sin(2\pi B_x x), \cos(2\pi B_x x)]. \quad (1.17)$$

Конкретные параметры вычислительного эксперимента, пространственно-временная область, длительность обучения и детали сравнения с точным решением приведены в главе 2.

Глава 2 Этап 1: базовый вычислительный эксперимент

2.1 Формирование обучающего множества и коллокация

Вычислительный эксперимент для решения уравнения Курамото–Сивашинского проводился в прямоугольной пространственно-временной области $(t, x) \in [0, 4] \times [-30, 30]$. Обучающее множество строилось по общей схеме, описанной в разделе 1.2: точки начального условия задавались на слое $t = 0$, точки граничных условий — на границах $x = -30$ и $x = 30$, а внутренние коллокационные точки распределялись внутри всей области. Во всех трёх группах использовалось случайное равномерное распределение в соответствующих подмножествах области определения. Схематически распределение точек показано на рисунок 1.

2.2 Параметры базовой модели

В качестве базовой модели использовалась полносвязная нейронная сеть $u_\theta(t, x)$ с нормированием выходной переменной и Фурье-кодированием входов, введёнными в разделе 1.2. Такое сочетание позволяло стабилизировать масштаб выходной функции и повысить способность сети передавать локальные пространственные особенности точного решения.

Обучение базовой модели проводилось до 13000 эпох. Для анализа динамики сходимости и эволюции аппроксимации дополнительно рассматривались промежуточные состояния после 3000, 7000 и 10000 эпох. Качество восстановления оценивалось сравнением с точным аналитическим решением на характерных временных срезах $t = 0$ и $t = 4$.

2.3 Анализ динамики обучения и эволюция аппроксимации

Графики рисунки 2–6 показывают типичную для PINN последовательность этапов обучения. На начальной стадии предсказание остаётся сглаженным и существенно отличается от точного решения как по амплитуде, так и по положению фронта. Такое поведение согласуется с доминированием

residual-компоненты, содержащей производную четвёртого порядка: на ранних итерациях сеть предпочитает низкочастотные профили с малой кривизной.

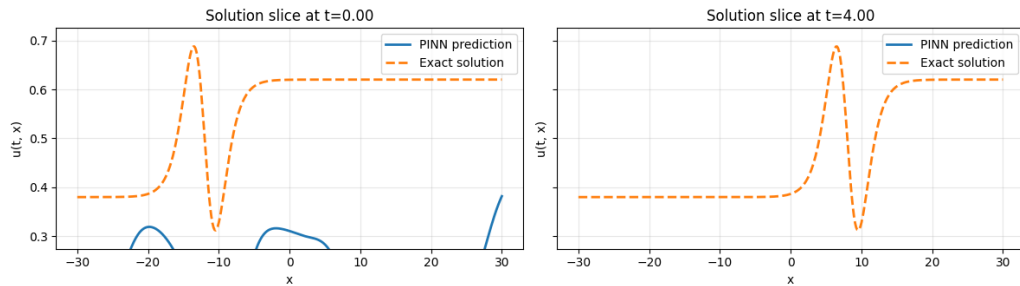


Рисунок 2 – Начальная эпоха

На промежуточных итерациях модель начинает воспроизводить глобальную геометрию бегущей волны: формируются основной пик и следующая за ним впадина, уменьшается ошибка на срезе $t = 0$, а профиль при $t = 4$ приобретает корректное направление переноса. В то же время сохраняются фазовый сдвиг и ошибка амплитуды, что указывает на неполное согласование между выполнением уравнения и воспроизведением временной эволюции решения.

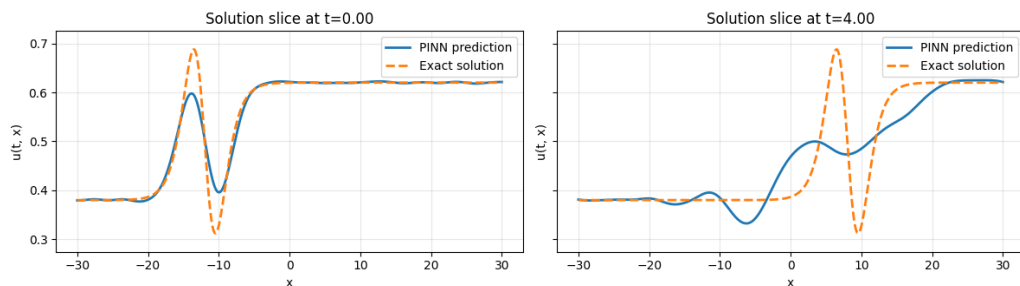


Рисунок 3 – 3000-я эпоха

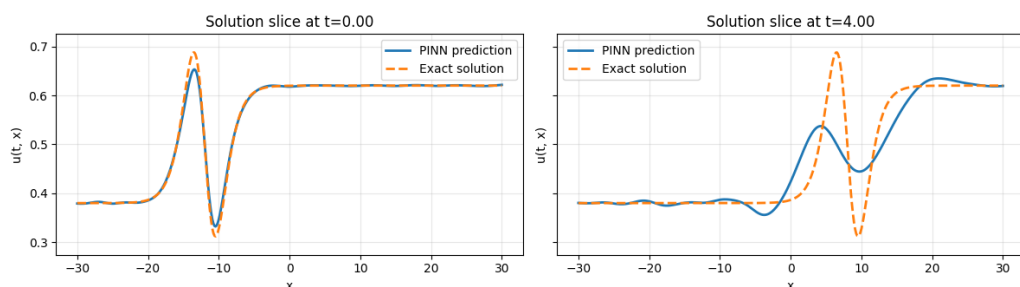


Рисунок 4 – 7000-я эпоха

На заключительной стадии предсказание практически совпадает с точным решением на рассматриваемых временных срезах. Модель корректно воспроизводит нелинейный профиль типа $\tanh / \tanh^3$, пространственный сдвиг волны и масштаб амплитуды.

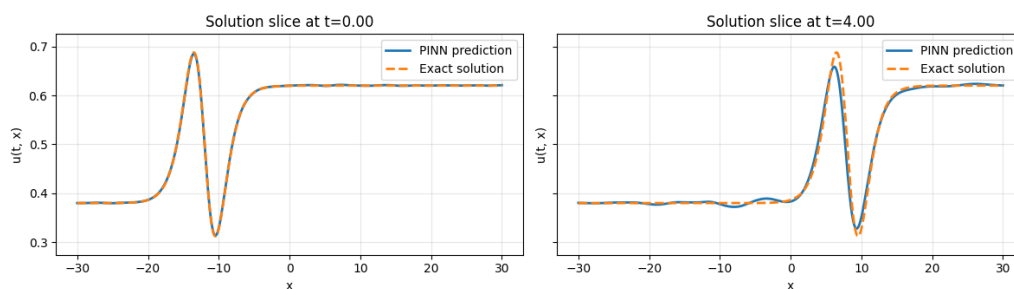


Рисунок 5 – 10000-я эпоха

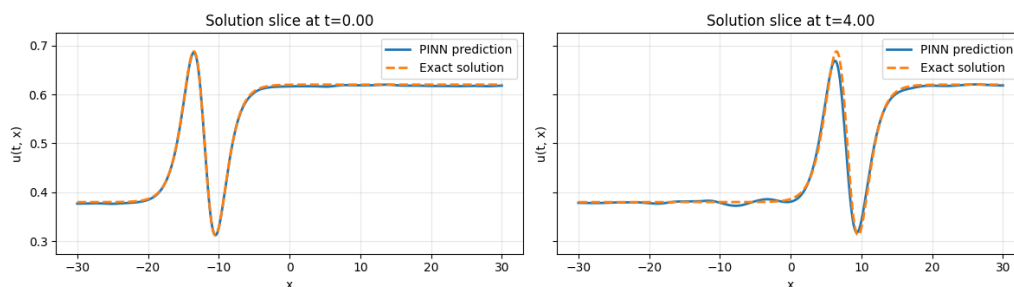


Рисунок 6 – 13000-я эпоха

2.4 Анализ ошибки

Как показывает рисунок 7, компоненты функции потерь убывают с различной скоростью. На ранних итерациях ошибка по начальному условию снижается быстрее, чем ошибка, связанная с граничными ограничениями. На поздней стадии компонент, соответствующий условиям Неймана, демонстрирует более ровную динамику и меньшую чувствительность к высокочастотным колебаниям, чем ошибка по начальному условию. В пределах данного эксперимента его можно рассматривать как более устойчивый ориентир при анализе сходимости отдельных частей функционала.

Суммарная функция потерь (рисунок 8) в целом демонстрирует хорошую сходимость: значение ошибки монотонно убывает на протяжении большей части процесса обучения и достигает порядка 10^{-5} , что соответствует качественной аппроксимации. На заключительном участке появляются локальные осцилляции, согласующиеся с ростом вклада одной из граничных компонент. Это указывает на дисбаланс между отдельными частями функционала и делает оправданным дополнительное перевзвешивание условий в функции потерь.

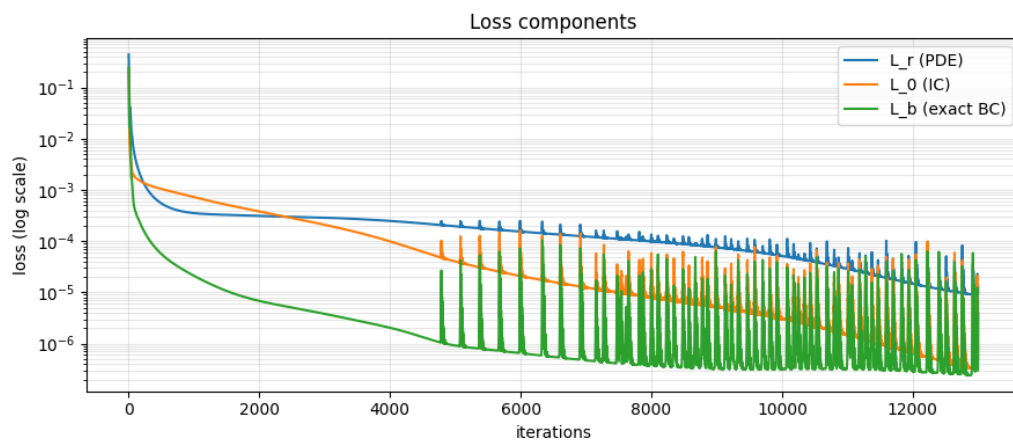


Рисунок 7 – Динамика компонент функции потерь, соответствующих различным условиям задачи

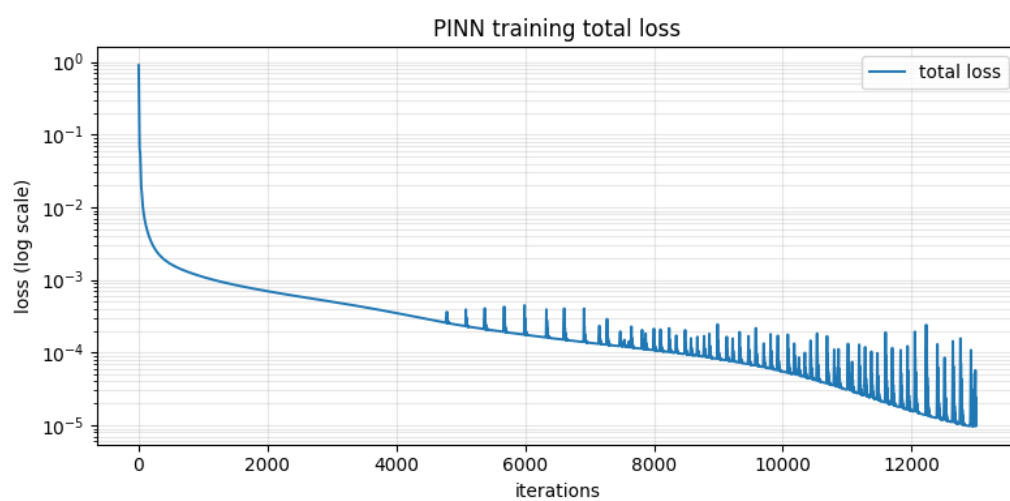


Рисунок 8 – Динамика суммарной функции потерь

Глава 3 Этап 2: понижение порядка и анализ деградации точности

3.1 Модификация постановки задачи и варианты архитектур PINN

На втором этапе проверялась гипотеза о том, что понижение порядка старшей пространственной производной может упростить обучение PINN. Для этого была введена дополнительная вспомогательная переменная формула (3.1):

$$v = \frac{\partial^2 u}{\partial x^2} \quad (3.1)$$

Данная подстановка преобразует исходное уравнение с производными четвёртого порядка в систему уравнений не выше второго порядка, задаваемую формулы (3.2) и (3.3):

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} + v + \frac{\partial^2 v}{\partial x^2} = 0. \quad (3.2)$$

$$v - \frac{\partial^2 u}{\partial x^2} = 0. \quad (3.3)$$

Мотивацией такой замены было исключение прямого вычисления производной четвёртого порядка, которое в PINN часто сопровождается жёсткостью градиентов и повышенной чувствительностью к гиперпараметрам. Вместе с тем новая постановка не сводится к упрощению оптимизации: она вводит дополнительное поле $v(t, x)$ и условие согласования формула (3.3), так что сеть должна одновременно восстановить $u(t, x)$, $v(t, x)$ и их взаимосвязь во всей области.

Для решения модифицированной системы были рассмотрены три архитектуры:

- **Единая нейронная сеть с двумя выходами (two_output):** Отображение пространства входов в двумерный выход (u, v) с общим скрытым представлением.
- **Две независимые нейронные сети (two_models):** Полное разделение параметров, где функции u и v аппроксимируются изолированными сетями.

- **Сеть с общим стволом и отдельными ветвями (shared_trunk):** Гибридный подход, при котором начальные слои извлекают общие физические закономерности, а независимые «головы» адаптируются к спецификам u и v .

3.2 Динамика обучения и сравнительный анализ архитектур

На рисунок 9 для всех архитектур второго этапа наблюдается одна и та же закономерность. На начальных итерациях total loss быстро убывает, затем после примерно $3 \cdot 10^3 - 4 \cdot 10^3$ шагов темп сходимости резко снижается, и траектории переходят в режим плато. Финальные значения total loss стабилизируются на уровне $1.4 \cdot 10^{-4} - 1.5 \cdot 10^{-4}$, тогда как базовая модель первого этапа достигает уровня порядка 10^{-5} . Следовательно, понижение порядка производной не устранило оптимизационный барьер, а перенесло его в новую постановку.

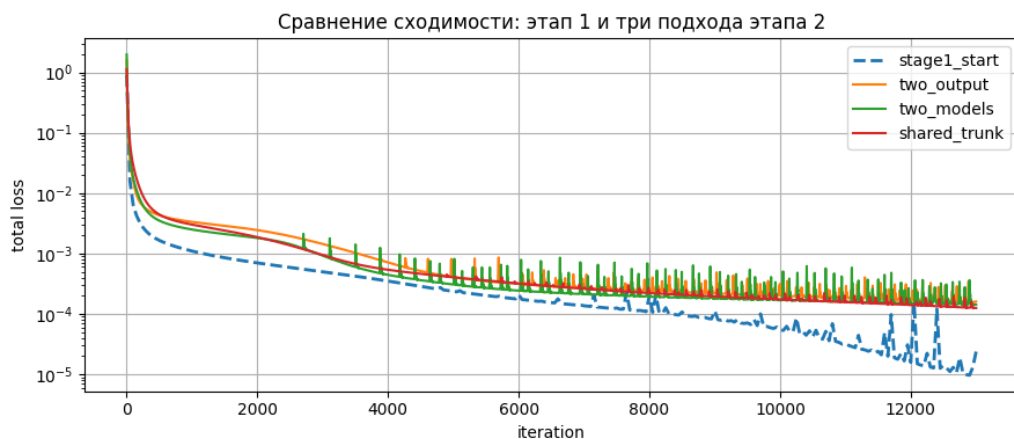


Рисунок 9 – Сравнение ошибок аппроксимации четырёх моделей

Сравнение кривых total loss на рисунки 10–12 показывает, что среди архитектур второго этапа наиболее ровную траекторию и наименьший финальный уровень ошибки демонстрирует shared_trunk. Архитектура two_output близка к ней по уровню ошибки, но сохраняет более регулярные всплески. Наиболее выраженные осцилляции наблюдаются у two_models, что указывает на наименьшую устойчивость этой схемы.

Графики компонент функции потерь на рисунки 13–15 показывают, что во всех трёх случаях ключевым ограничением остаётся невязка уравнения L_r . Компоненты начального условия L_0 и граничных условий L_b убывают быстрее и к концу обучения оказываются на один–два порядка ниже. Тем самым

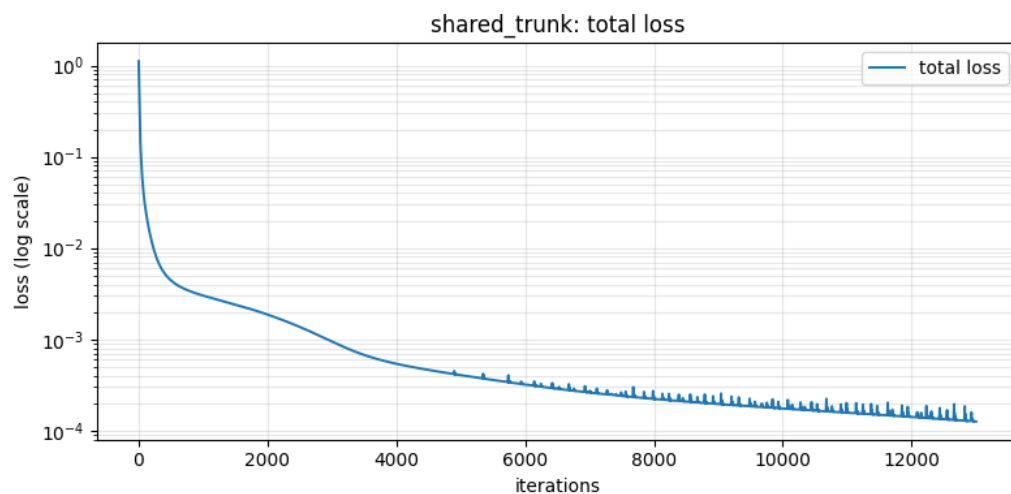


Рисунок 10 – Общая ошибка гибридной модели с общим стволом

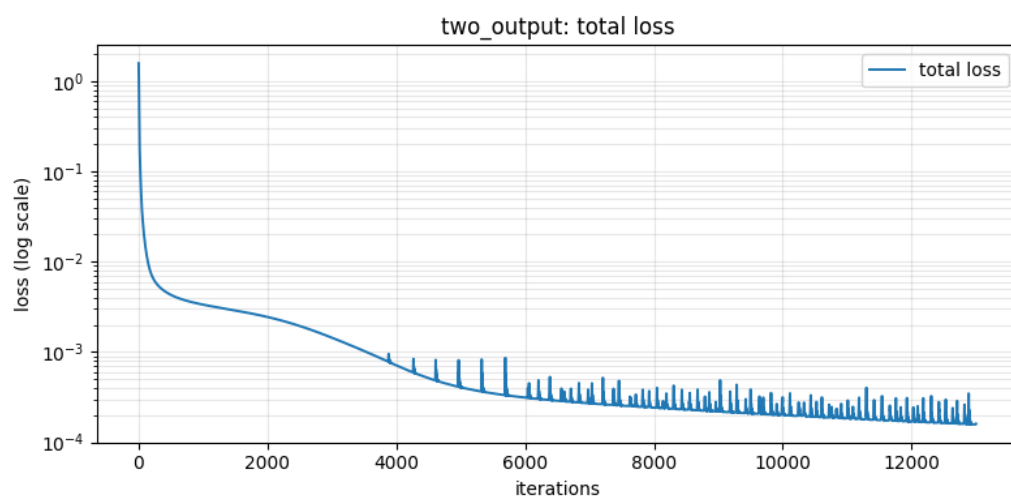


Рисунок 11 – Общая ошибка модели с двумя выходами

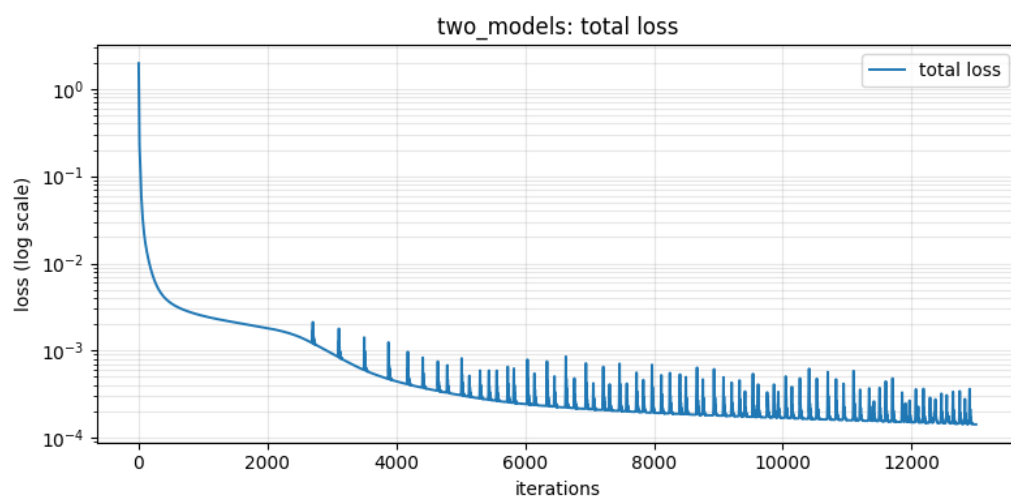


Рисунок 12 – Общая ошибка модели с двумя независимыми нейронными сетями

общая закономерность обучения моделей второго этапа состоит в следующем: начальное снижение total loss достигается за счёт быстрого уменьшения ошибок по данным и граничным ограничениям, после чего дальнейшая сходимость определяется residual-компонентой и необходимостью согласования полей u и v .

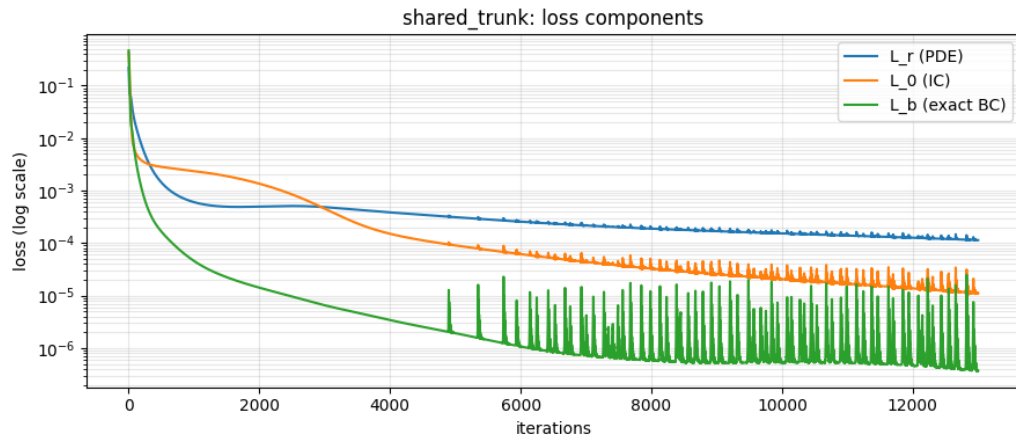


Рисунок 13 – Компоненты функции потерь гибридной модели с общим стволом

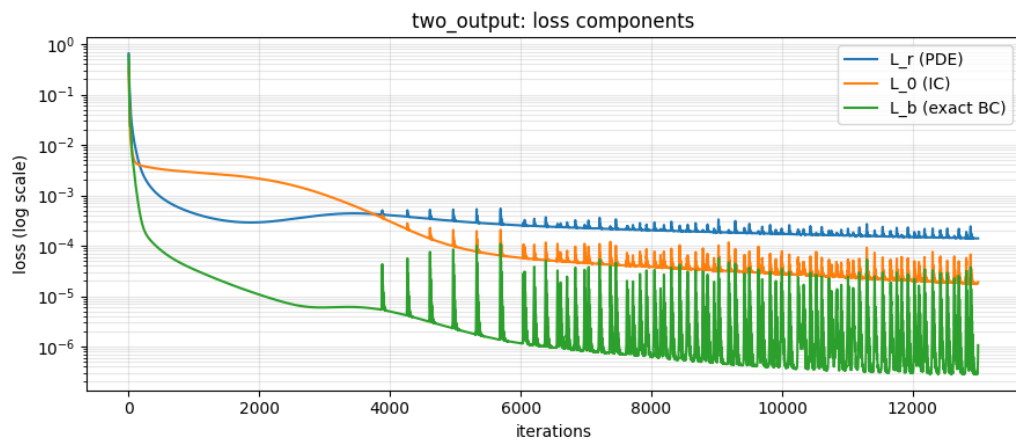


Рисунок 14 – Компоненты функции потерь модели с двумя выходами

Пики на кривых total loss и отдельных компонент возникают при попытках дальнейшего уменьшения L_r после выхода на плато. Их наибольшая амплитуда в two_models согласуется с тем, что две независимые сети должны синхронизироваться только через функционал потерь. В two_output этот конфликт выражен слабее, однако общее скрытое представление должно одновременно кодировать поля разной гладкости и масштаба. В shared_trunk общие пространственно-временные признаки извлекаются совместно, а различия между u и v частично компенсируются отдельными выходными ветвями, что соответствует более ровкой динамике.

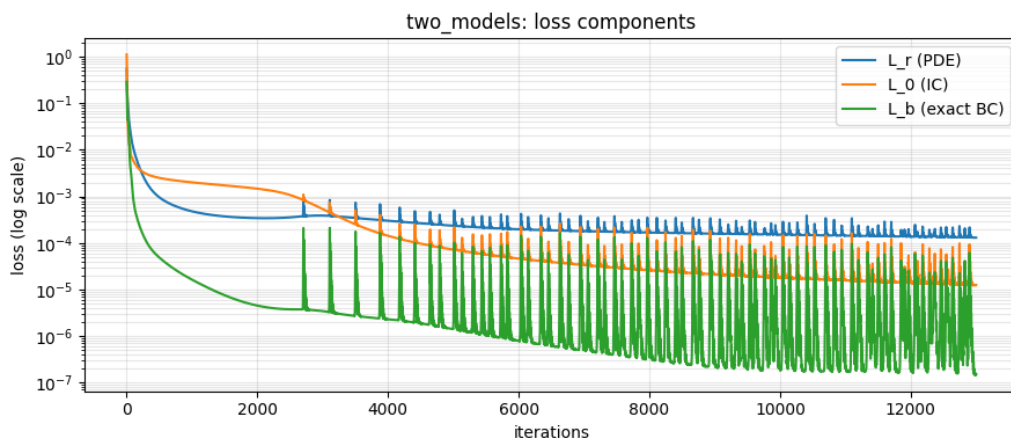


Рисунок 15 – Компоненты функции потерь модели с двумя независимыми нейронными сетями

Эволюция аппроксимации на рисунки 16–18 показывает, что во всех архитектурах поле $u(t, x)$ восстанавливается заметно лучше, чем поле $v(t, x)$. На срезе $t = 0$ все три модели после обучения передают основную структуру фронта. На срезах $t = 2$ и $t = 4$ различия становятся более выраженными: `shared_trunk` лучше сохраняет положение фронта и амплитуду, `two_output` занимает промежуточное положение, а `two_models` демонстрирует наиболее заметное сглаживание переходных зон.

Для поля $v(t, x)$ ни одна из архитектур не воспроизводит с той же точностью узкие знакопеременные пики точного решения на поздних временных срезах. Предсказания остаются сглаженными и имеют заниженную амплитуду, что указывает на трудности восстановления локальной кривизны, связанной с условием $v = u_{xx}$. Та же закономерность отражается и на сводном графике ошибки: минимальную относительную ошибку среди архитектур второго этапа демонстрирует `shared_trunk`, за ним следует `two_output`, тогда как `two_models` показывает наихудший результат.

3.3 Причины деградации точности

Наблюдаемая деградация точности связана со структурой новой задачи оптимизации. Понижение порядка производной устраняет прямое вычисление u_{xxxx} , но заменяет его одновременным обучением двух полей с различной гладкостью. Поле $u(t, x)$ задаёт сравнительно гладкий бегущий фронт, тогда как $v(t, x) = u_{xx}(t, x)$ содержит более локальные и знакопеременные структуры. Поэтому малое локальное отклонение в u усиливается как в аппроксимации v ,

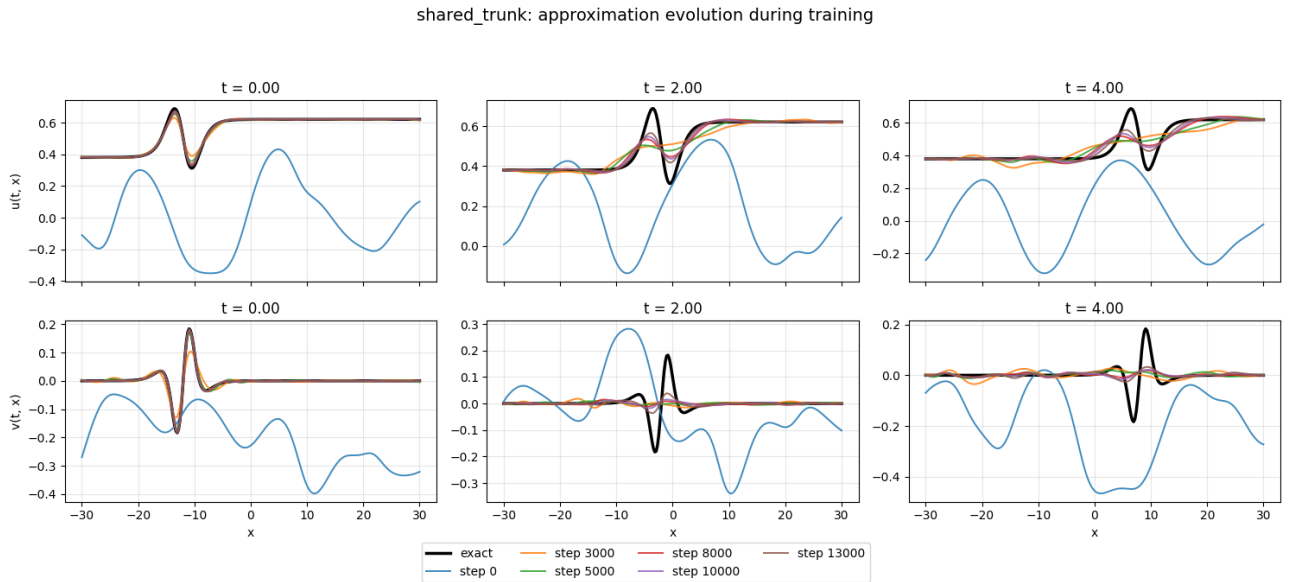


Рисунок 16 – Эволюция аппроксимации гибридной модели с общим стволом

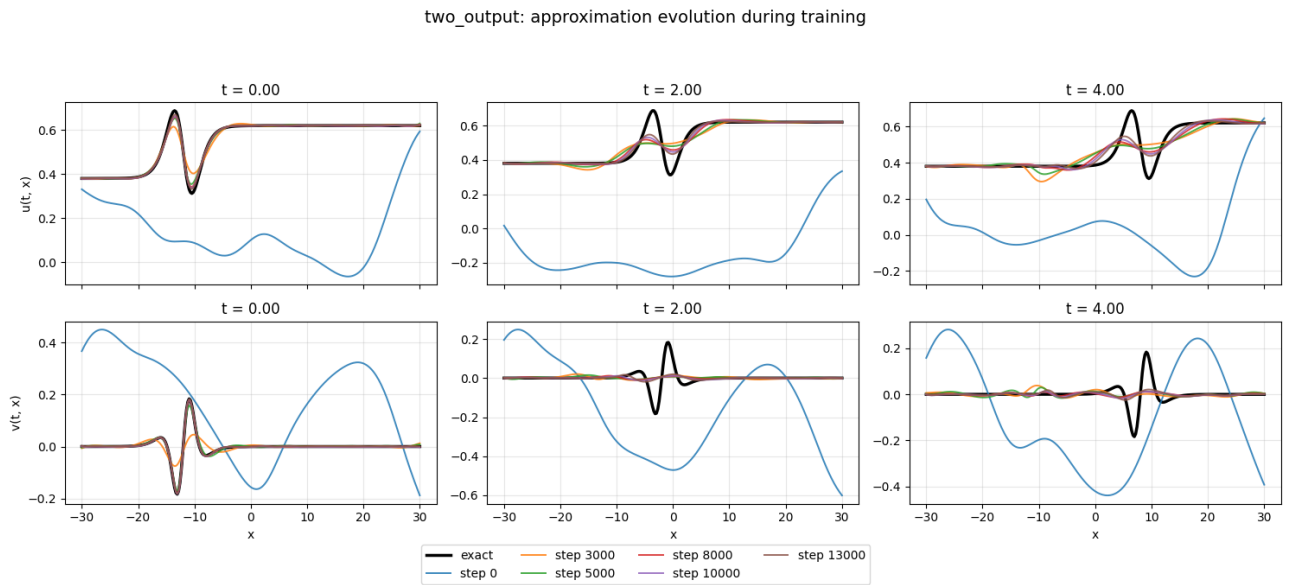


Рисунок 17 – Эволюция аппроксимации модели с двумя выходами

так и в невязке условия согласования формула (3.3).

Эта структура согласуется с общей динамикой обучения. На ранней стадии уменьшение total loss обеспечивается главным образом за счёт снижения ошибок по начальному и граничным условиям и за счёт восстановления грубой формы поля u . После этого дальнейшее уменьшение ошибки требует одновременного снижения residual-компоненты и невязки согласования между u и v , что и приводит к формированию плато.

Архитектурные различия определяются тем, как каждая схема разрешает эту многомасштабную связь. В `two_models` взаимодействие между u и v

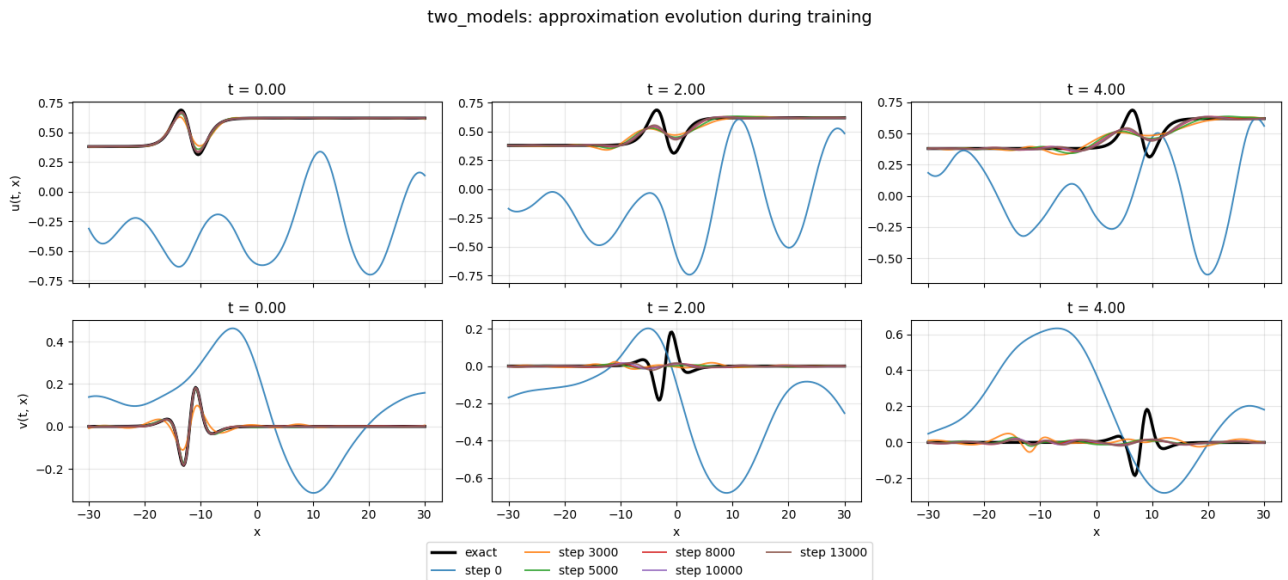


Рисунок 18 – Эволюция аппроксимации модели с двумя независимыми нейронными сетями

реализуется только через функционал потерь, поэтому обновление одной сети меняет целевую конфигурацию для другой; это соответствует наиболее выраженным осцилляциям. В `two_output` конфликт ослаблен общим скрытым представлением, но оно должно одновременно кодировать поля разного масштаба. Архитектура `shared_trunk` обеспечивает компромисс между совместным извлечением общих признаков и специализацией выходов, что согласуется с меньшей амплитудой колебаний и более низкой итоговой ошибкой [6; 7].

3.4 Выводы: деградация эффективности при усложнении системы

Результаты второго этапа допускают три основных вывода:

1. Для всех архитектур второго этапа характерно быстрое начальное снижение `total loss`, после которого обучение переходит в режим плато. Лимитирующим фактором при этом становится `residual`-компонента и связанное с ней согласование полей u и v .
2. Понижение порядка за счёт введения переменной v не привело к улучшению точности по сравнению с прямой аппроксимацией исходного уравнения четвёртого порядка. Финальные ошибки моделей второго этапа

остаются примерно на порядок выше, чем у базовой модели первого этапа.

3. Среди рассмотренных архитектур наименьшую итоговую ошибку и наиболее устойчивую динамику обучения демонстрирует `shared_trunk`; архитектура `two_output` занимает промежуточное положение, а `two_models` оказывается наименее устойчивой.

С учётом наблюдаемого дисбаланса между компонентами функции потерь для данной постановки оправдано адаптивное перевзвешивание `residual`- и `boundary`-компонент, а также отдельное масштабирование выходов u и v .

Глава 4 Анализ Neural Tangent Kernel для сети 1 первого этапа

4.1 Описание нейронного касательного ядра

Neural Tangent Kernel (NTK) представляет собой ядро, порожденное нейронной сетью через градиенты выхода сети по её параметрам. В современной теории глубокого обучения NTK играет важную роль, поскольку позволяет описывать обучение нейронной сети в функциональном пространстве, а не только в пространстве параметров. Это особенно важно в режиме большой ширины, когда динамика градиентного спуска приобретает почти линейный характер и становится близкой к обучению ядрового метода.

Ключевая идея состоит в том, что при достаточно большой ширине скрытых слоёв случайно инициализированная сеть ведёт себя почти как линейная модель относительно параметров в окрестности точки инициализации. В этом пределе NTK стремится к детерминированному ядру, а обучение сети по градиентному спуску оказывается эквивалентным kernel regression с данным ядром. Поэтому NTK служит мостом между теорией глубоких сетей, гауссовскими процессами и классическими ядровыми методами.

4.2 Математическое обоснование

4.2.1 Определение NTK

Пусть нейронная сеть задаёт скалярную функцию

$$f(x, \theta) : \mathbb{R}^d \rightarrow \mathbb{R}, \quad (4.1)$$

где x — вход, а $\theta \in \mathbb{R}^p$ — вектор параметров сети. Для двух входов x и x' Neural Tangent Kernel определяется как скалярное произведение градиентов выхода сети по параметрам:

$$K(x, x'; \theta) = \nabla_{\theta} f(x, \theta)^{\top} \nabla_{\theta} f(x', \theta). \quad (4.2)$$

Если рассматривать набор точек $X = \{x_i\}_{i=1}^n$, то соответствующая матрица

NTK имеет вид

$$\Theta_{ij}(\theta) = K(x_i, x_j; \theta) = \nabla_{\theta} f(x_i, \theta)^{\top} \nabla_{\theta} f(x_j, \theta). \quad (4.3)$$

Иначе говоря, NTK измеряет сходство двух входов не в исходном пространстве признаков, а в пространстве чувствительности модели к своим параметрам.

4.2.2 Переход к линейной динамике обучения

Рассмотрим обучение по функции потерь среднеквадратичного отклонения

$$\mathcal{L}(\theta) = \frac{1}{2} \sum_{i=1}^n (f(x_i, \theta) - y_i)^2. \quad (4.4)$$

В режиме непрерывного времени градиентный спуск записывается как gradient flow:

$$\frac{d\theta}{dt} = -\nabla_{\theta} \mathcal{L}(\theta) = -\sum_{j=1}^n (f(x_j, \theta) - y_j) \nabla_{\theta} f(x_j, \theta). \quad (4.5)$$

Обозначим через

$$u_i(t) = f(x_i, \theta(t)), \quad u(t) = (u_1(t), \dots, u_n(t))^{\top}. \quad (4.6)$$

Тогда по правилу цепочки

$$\frac{du_i}{dt} = \nabla_{\theta} f(x_i, \theta)^{\top} \frac{d\theta}{dt}. \quad (4.7)$$

Подставляя (4.5), получаем

$$\frac{du_i}{dt} = -\sum_{j=1}^n \Theta_{ij}(\theta) (u_j(t) - y_j). \quad (4.8)$$

В матричной форме это даёт

$$\frac{du}{dt} = -\Theta(\theta(t)) (u(t) - y). \quad (4.9)$$

Формула (4.9) показывает, что именно матрица NTK определяет скорость уменьшения ошибки по различным направлениям в пространстве функций.

Если в ходе обучения ядро меняется мало, то можно считать

$$\Theta(\theta(t)) \approx \Theta(\theta_0) = \Theta_0, \quad (4.10)$$

и тогда динамика становится линейной:

$$\frac{du}{dt} = -\Theta_0(u(t) - y). \quad (4.11)$$

Решение этой системы имеет вид

$$u(t) - y = \exp(-\Theta_0 t)(u(0) - y). \quad (4.12)$$

Таким образом, компоненты ошибки, соответствующие большим собственным значениям НТК, затухают быстрее, тогда как направления, связанные с малыми собственными значениями, обучаются медленнее.

4.2.3 Связь регрессией и динамика НТК в пределе бесконечной ширины

В режиме линеаризованного (так называемого «ленивого») обучения динамика нейронной сети большой ширины аналитически эквивалентна методам ядерной регрессии. Если процесс оптимизации продолжается до полной интерполяции данных обучающей выборки при условии квазистационарности НТК, предельное решение для произвольного входного вектора x принимает следующую форму:

$$f_\infty(x) = f_0(x) + \Theta(x, X)\Theta(X, X)^{-1}(y - f_0(X)), \quad (4.13)$$

где f_0 — аппроксимирующая функция в момент случайной инициализации параметров, $\Theta(x, X)$ — вектор значений НТК между точкой x и объектами обучающей выборки, а $\Theta(X, X)$ — матрица Грама НТК.

Матрица Грама $\Theta(X, X) \in \mathbb{R}^{N \times N}$ представляет собой симметричную положительно полуопределенную матрицу, где N — количество обучающих примеров. Каждый её элемент (i, j) определяется как скалярное произведение градиентов выхода сети по вектору параметров θ , вычисленных в точках x_i и x_j :

$$\Theta_{ij} = \langle \nabla_\theta f(x_i, \theta), \nabla_\theta f(x_j, \theta) \rangle. \quad (4.14)$$

Визуально матрица Грама характеризует геометрию признакового

пространства, порожденное архитектурой сети: диагональные элементы отражают «чувствительность» сети к отдельным объектам, а внедиагональные — степень информационного перекрытия и взаимного влияния пар объектов при обновлении параметров.

Устойчивость НТК при увеличении ширины слоёв $m \rightarrow \infty$ обосновывается законом больших чисел. Поскольку вклад каждого отдельного нейрона в итоговый результат становится исчезающе малым, совокупная матрица НТК при инициализации концентрируется около своего детерминированного предела:

$$\Theta(\theta_0) \xrightarrow{m \rightarrow \infty} \Theta_\infty. \quad (4.15)$$

При выборе адекватной параметризации изменения параметров в процессе обучения имеют порядок, недостаточный для значимого изменения ядра. Следовательно, на конечном временном интервале выполняется приближенное равенство $\Theta(\theta(t)) \approx \Theta_\infty$, что обуславливает переход динамики системы в режим, близкий к линейному.

С содержательной точки зрения НТК служит мерой подобия откликов сети. Если градиенты $\nabla_\theta f(x, \theta)$ и $\nabla_\theta f(x', \theta)$ характеризуются высокой степенью сонаправленности, то минимизация ошибки в точке x приведет к синхронному изменению предсказания в точке x' . Таким образом, НТК можно интерпретировать как оператор распространения информации, определяющий, как локальное исправление ошибки на одном объекте транслируется на всю выборку.

4.3 Спектральные и алгебраические свойства НТК

Матрица НТК обладает рядом фундаментальных свойств, определяющих характер оптимизации и обобщающую способность нейронных сетей в режиме бесконечной ширины.

На основе аналитического определения НТК как скалярного произведения градиентов функции по параметрам, матрица $\Theta(X, X)$ для любой конечной выборки $X = \{x_i\}_{i=1}^n$ является симметричной и положительно полуопределенной. Свойство симметричности $K(x, x'; \theta) = K(x', x; \theta)$ следует из коммутативности скалярного произведения. Положительная полуопределенность подтверждается анализом квадратичной формы для

произвольного вектора коэффициентов $c \in \mathbb{R}^n$:

$$c^\top \Theta c = \sum_{i,j=1}^n c_i c_j \nabla_\theta f(x_i, \theta)^\top \nabla_\theta f(x_j, \theta) = \left\| \sum_{i=1}^n c_i \nabla_\theta f(x_i, \theta) \right\|_2^2 \geq 0. \quad (4.16)$$

Из этого следует, что в условиях точной арифметики все собственные значения λ_i матрицы НТК неотрицательны. Появление незначительных отрицательных значений в практических численных экспериментах обычно интерпретируется как следствие погрешностей машинного округления.

Спектральное разложение матрицы $\Theta = Q \Lambda Q^\top$ (где $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$ и $\lambda_1 \geq \lambda_2 \geq \dots \geq 0$) позволяет оценить количество независимых направлений изменения аппроксимирующей функции. Скорость диссипации (затухания) спектра определяет эффективную сложность модели: быстрое убывание собственных значений указывает на доминирование низкоразмерного подпространства в процессе обучения. Для количественного описания структуры матрицы используются след ($\text{trace}(\Theta)$), ранг ($\text{rank}(\Theta)$) и эффективный ранг (r_{eff}), вычисляемый через энтропию нормированного спектра:

$$p_i = \frac{\lambda_i}{\sum_j \lambda_j}, \quad r_{\text{eff}} = \exp \left(- \sum_i p_i \ln p_i \right). \quad (4.17)$$

Эффективный ранг служит мерой информационной емкости модели, отражая реальное число степеней свободы, задействованных при подгонке данных.

4.3.1 Обусловленность и устойчивость динамики

Важнейшим дескриптором устойчивости процесса обучения является число обусловленности, определяемое как отношение максимального собственного значения к минимальному положительному:

$$\kappa(\Theta) = \frac{\lambda_{\max}}{\lambda_{\min}^+}. \quad (4.18)$$

Высокое значение $\kappa(\Theta)$ свидетельствует о плохой обусловленности системы, что влечет за собой существенную неоднородность скоростей сходимости: компоненты ошибки, соответствующие старшим собственным числам, подавляются экспоненциально быстро, в то время как компоненты, связанные с хвостом спектра, требуют длительного времени для коррекции. Такая «жесткая»

динамика повышает чувствительность модели к шумовым компонентам выборки и снижает стабильность численных алгоритмов оптимизации.

4.4 Практический пример: вычисление NTK для MLP в TensorFlow

Для практической демонстрации рассмотрим полносвязную сеть (MLP) с двумя скрытыми слоями и нелинейностью $\tanh$. На конечной выборке $\{x_i\}_{i=1}^n$ эмпирический NTK вычисляется следующим образом:

1. для каждого входа x_i вычисляется градиент $\nabla_{\theta} f(x_i, \theta)$;
2. градиенты векторизуются и собираются в матрицу Якоби $J \in \mathbb{R}^{n \times p}$;
3. матрица NTK строится как

$$\Theta = JJ^{\top}. \quad (4.19)$$

В данной работе все вычисления проводились в TensorFlow. Для фиксированной выборки эмпирическая матрица NTK строилась как на инициализации, так и после завершения обучения MLP, после чего по ней вычислялись спектр собственных значений, ранг, effective rank и condition number. На основе этих данных были получены тепловые карты NTK до и после обучения, PCA-проекция строк матрицы и кластеризованная матрица, что позволило сопоставить спектральные характеристики ядра с геометрией данных и с наблюдаемой динамикой оптимизации.

4.5 Визуализации и анализ результатов

В данном разделе представлены визуализации эмпирического нейрокасательного ядра (NTK) для модельной задачи одномерной регрессии, полученные с использованием библиотеки TensorFlow.

Матрица Грама до обучения (рисунок 19) обладает сглаженной и слабо контрастной структурой. Это свидетельствует о том, что на начальном этапе попарные взаимодействия объектов определяются преимущественно индуктивным смещением архитектуры и случайной инициализацией весов, а сеть еще не сформировала избирательную геометрию связей. После

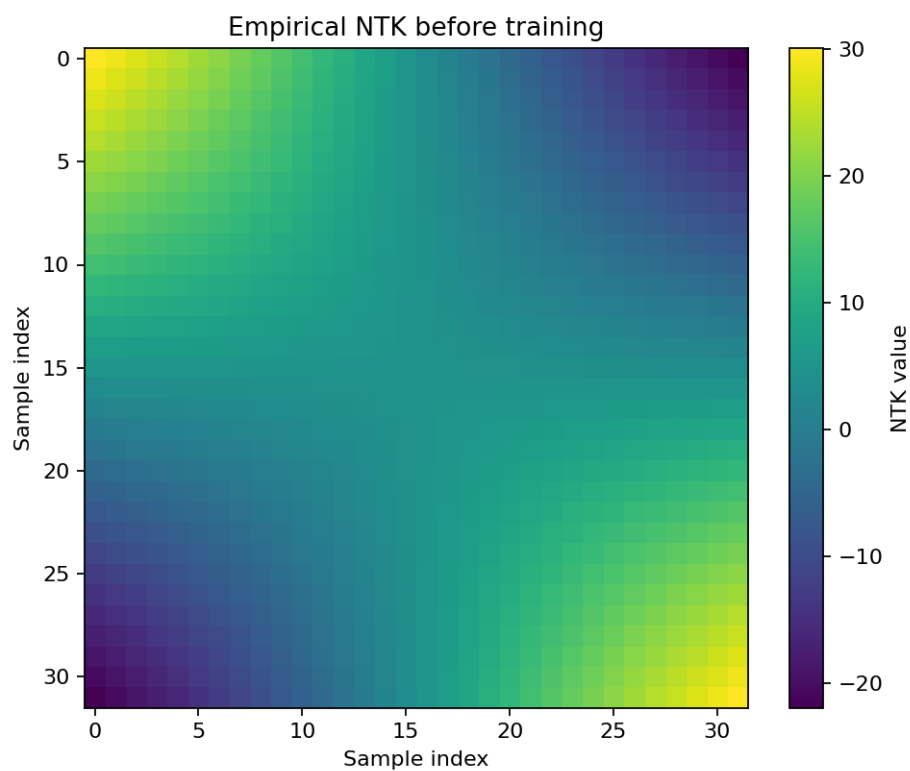


Рисунок 19 – Эмпирическая матрица NTK до обучения.

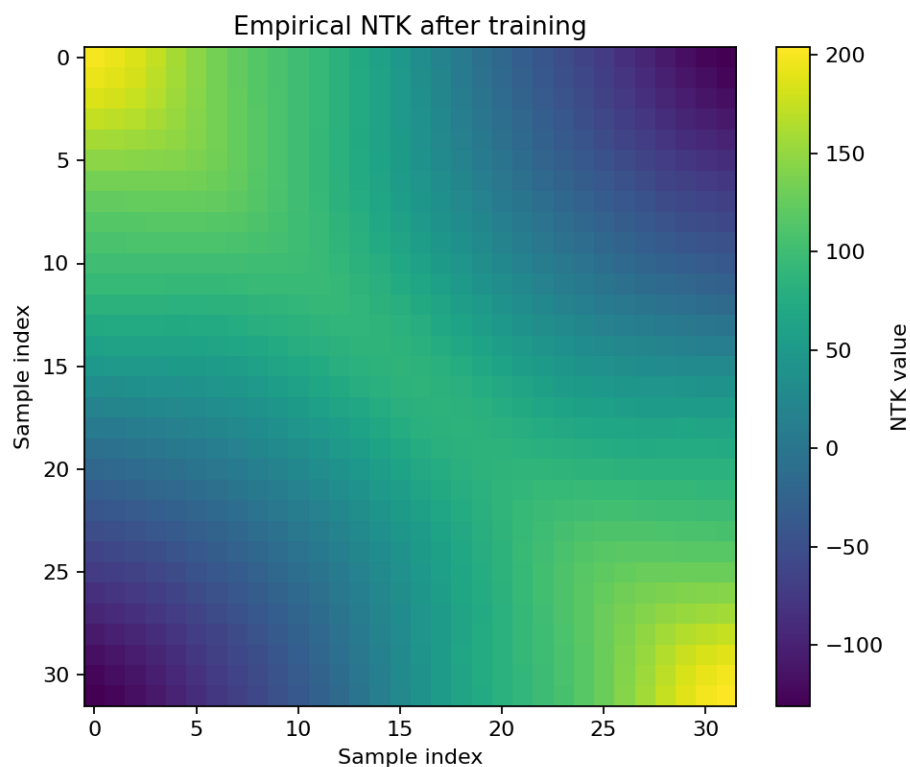


Рисунок 20 – Эмпирическая матрица NTK после обучения на задаче одномерной регрессии.

обучения (рисунок 20) структура ядра становится гетерогенной. Высокие значения на главной диагонали указывают на высокую чувствительность

предсказаний к собственным градиентным направлениям. Внедиагональные элементы демонстрируют, что геометрически близкие входы обучаются согласованно: перенос информации между соседними точками усиливается, а для удаленных областей — ослабляется. Таким образом, индуцированная метрика адаптируется к внутренней структуре данных.

При анализе тепловых карт НТК следует учитывать, что элемент Θ_{ij} отражает меру согласованности градиентов для объектов x_i и x_j . Интенсивная диагональ соответствует высокой норме собственных градиентных откликов, а внедиагональные области — наличию функциональной связи. При упорядочивании индексов согласно геометрии выборки светлые блоки интерпретируются как кластеры согласованного обновления параметров, тогда как темные участки указывают на декорреляцию.

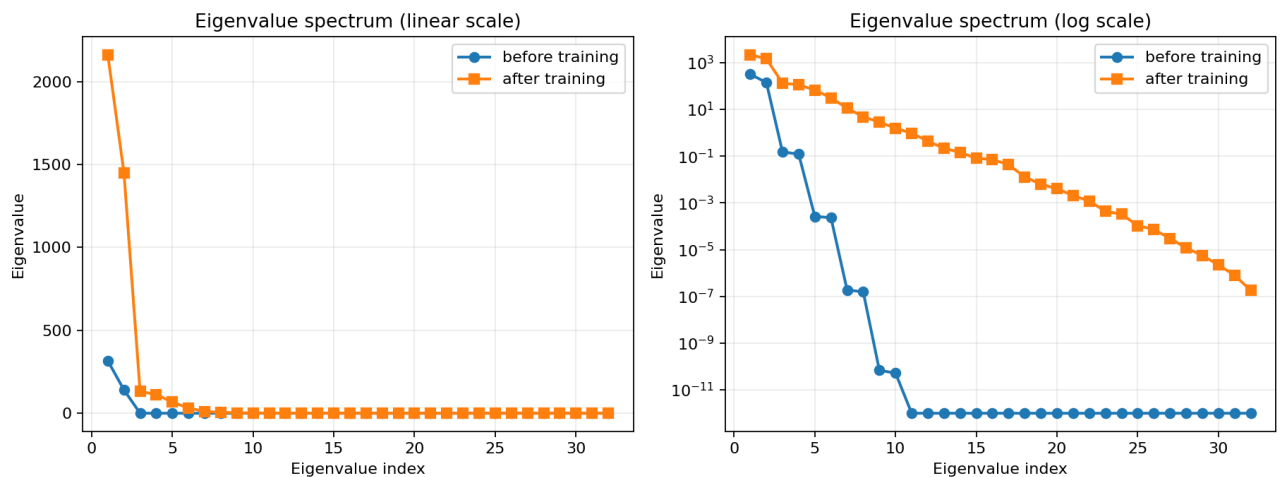


Рисунок 21 – Спектр собственных значений НТК до и после обучения (слева — линейный масштаб, справа — логарифмический).

Анализ спектра собственных значений НТК (рисунок 21) демонстрирует, что в процессе обучения доминирующие значения возрастают, а сам спектр убывает медленнее. Это указывает на повышение чувствительности модели к главным направлениям в данных и увеличение числа задействованных мод обучения. Слишком быстрое убывание спектра свидетельствовало бы о вырождении ядра и существенном ограничении выразительной способности сети.

Прямое сравнение матриц (рисунок 22) подтверждает, что базовая глобальная организация ядра сохраняется, однако локальные взаимодействия становятся значительно более выраженными. Обучение перераспределяет веса связей, повышая разрешающую способность НТК в областях, критичных для аппроксимации целевой функции. Ядро сети конечной ширины не остается

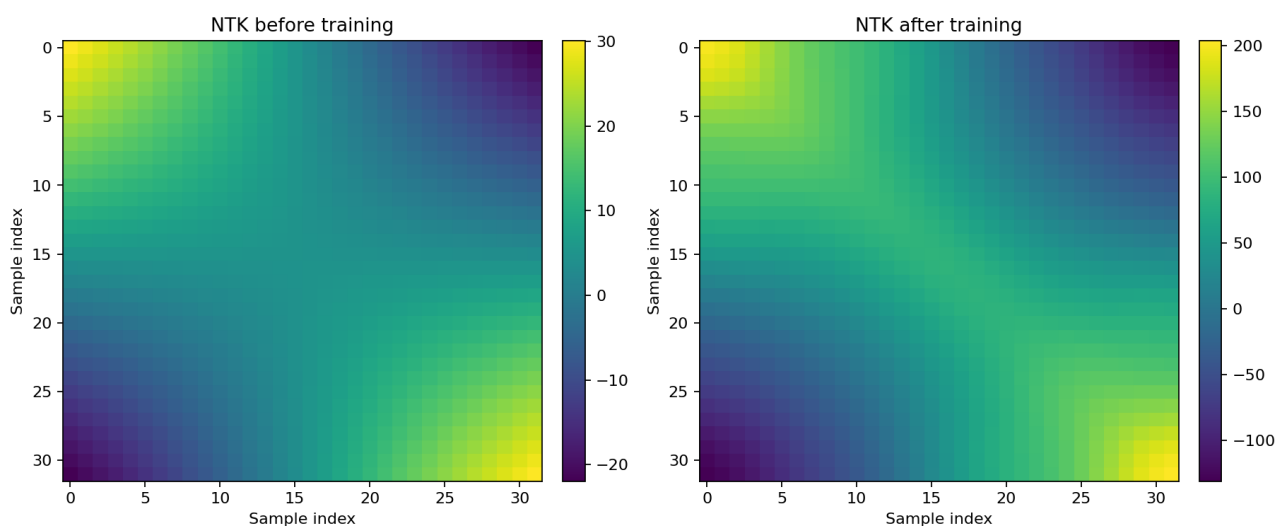


Рисунок 22 – Сравнение глобальной и локальной структуры матриц NTK до и после обучения.

статичным, а эволюционирует, отражая адаптацию признакового пространства модели.

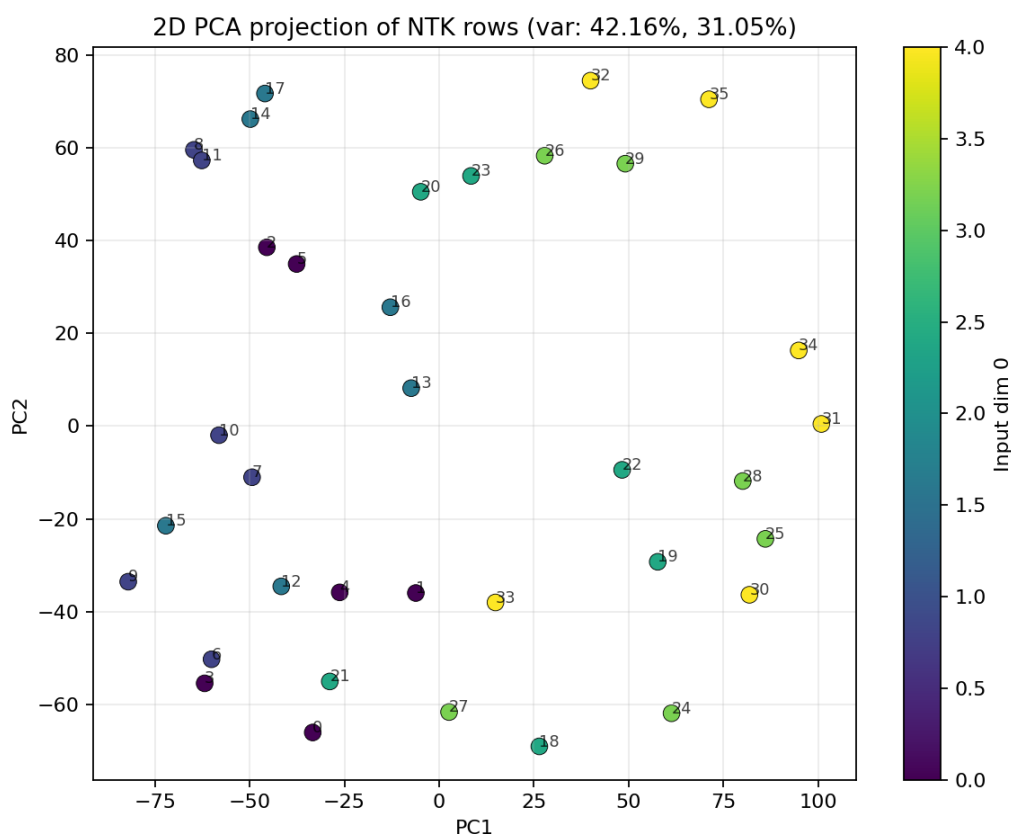


Рисунок 23 – PCA-проекция строк матрицы NTK.

Для перехода от матричного представления к геометрической интерпретации применена проекция строк матрицы NTK методом главных компонент (рисунок 23). Поскольку каждый вектор описывает профиль

взаимодействия отдельного объекта со всей выборкой, компактные кластеры на плоскости соответствуют объектам со схожей градиентной геометрией. Заметный разброс точек выявляет зоны непрерывного изменения локальной структуры.

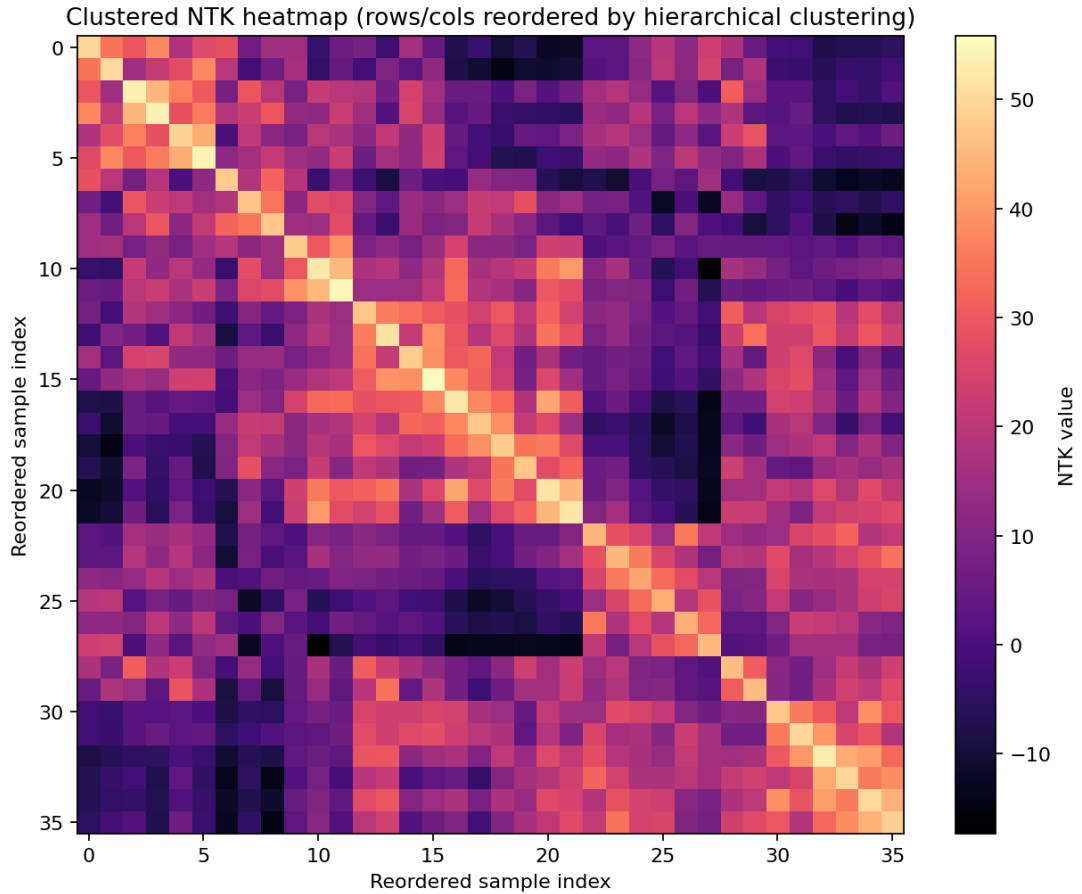


Рисунок 24 – Матрица NTK после иерархической кластеризации строк и столбцов.

Иерархическая кластеризация матрицы (рисунок 24) подтверждает этот вывод: после перестановки строк и столбцов отчетливо выделяются группы объектов с интенсивными внутрикластерными связями. В индуцированном функциональном пространстве данные образуют систему подмножеств с близкой геометрией градиентов, соединенных областями плавных переходов, о чем свидетельствуют ненулевые межкластерные элементы.

Карта чувствительности обучаемых блоков сети (рисунок 25) иллюстрирует вклад различных параметров в формирование ядра. В отличие от стандартной матрицы Грама, здесь по строкам отложены объекты выборки, а по столбцам — обучаемые слои модели. Интенсивность цвета отражает норму локального градиента $\|\partial f(x_i)/\partial \theta_\ell\|_2$. Светлые вертикальные полосы обозначают слои,

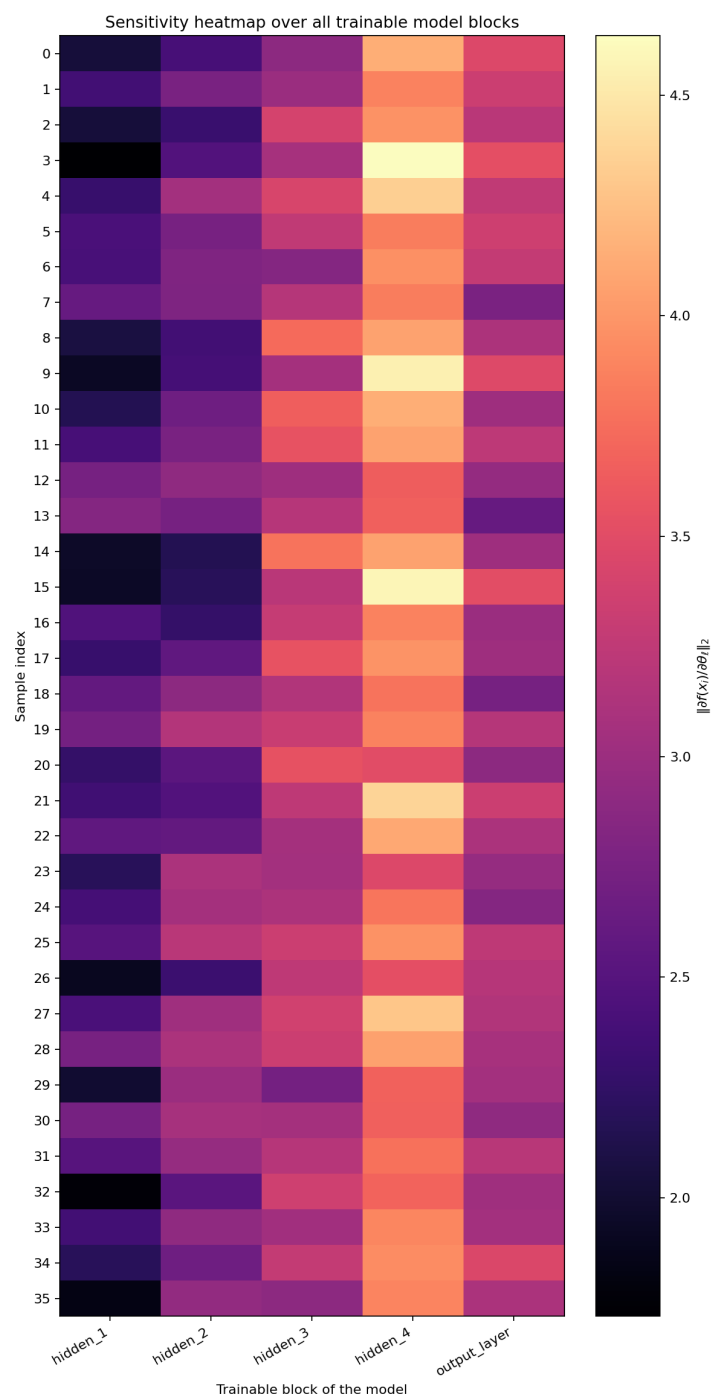


Рисунок 25 – Карта градиентной чувствительности обучаемых блоков модели.

вносящие наибольший усредненный вклад в чувствительность. Сравнение значений по строкам позволяет определить доминирующие слои для конкретного входа, а по столбцам — оценить глобальную однородность реакции отдельного блока. Локализованная высокая интенсивность в столбце указывает на избирательную адаптацию слоя к определенному сегменту входного пространства.

4.6 Интерпретация результатов

Анализ эмпирического нейронного касательного ядра (НТК) подтверждает адаптацию модели к структуре данных в процессе оптимизации. Если до обучения ядро отражает лишь индуктивное смещение архитектуры, то после завершения процесса наблюдается концентрация взаимодействий в окрестностях геометрически близких точек. Усиление диагонали и появление локальных блоков на тепловых картах (рисунок 20) свидетельствуют о переходе к менее изотропной метрике, согласованной с топологией входов.

Спектральные характеристики указывают на усложнение структуры ядра: ранг матрицы возрос с 7 до 20, а эффективный ранг — с 2.14 до 2.95. Рост ведущих собственных значений (рисунок 21) ускоряет освоение наиболее информативных мод. Эти данные коррелируют с результатами PCA-проекции строк (рисунок 23) и иерархической кластеризации (рисунок 24), которые визуализируют разделение объектов на группы с однородными градиентными профилями.

Обогащение структуры сопровождается ростом числа обусловленности до $7.94 \cdot 10^8$. Высокая чувствительность хвоста спектра к возмущениям объясняет появление малых отрицательных значений как вычислительных артефактов, не нарушающих положительную полуопределенность ядра. При этом анализ чувствительности блоков (рисунок 25) выявил согласованный вклад нескольких слоев в формирование НТК, что обеспечивает избирательную адаптацию модели к различным областям входного пространства.

С точки зрения обобщающей способности, проекция полезного сигнала на ведущие собственные векторы и локализация взаимодействий способствуют эффективному обучению. Зафиксированная эволюция ранга и связности позволяет классифицировать режим работы сети как промежуточный: модель сохраняет глобальную организацию ядрового представления, но демонстрирует нетривиальную адаптацию признаков (feature learning), выходящую за рамки режима «ленивого обучения».

Список литературы

1. Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains / M. Tancik [и др.] // *Advances in Neural Information Processing Systems* 33. — 2020. — С. 7537—7547.
2. *Kuramoto Y.* Diffusion-Induced Chaos in Reaction Systems // *Progress of Theoretical Physics Supplement*. — 1978. — Т. 64. — С. 346—367.
3. On the Spectral Bias of Neural Networks / N. Rahaman [и др.] // *Proceedings of the 36th International Conference on Machine Learning*. Т. 97. — 2019. — С. 5301—5310. — (Proceedings of Machine Learning Research).
4. *Raissi M., Perdikaris P., Karniadakis G. E.* Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Nonlinear Partial Differential Equations // *Journal of Computational Physics*. — 2019. — Т. 378. — С. 686—707. — DOI: 10.1016/j.jcp.2018.10.045.
5. *Sivashinsky G. I.* Nonlinear Analysis of Hydrodynamic Instability in Laminar Flames—I. Derivation of Basic Equations // *Acta Astronautica*. — 1977. — Т. 4, № 11/12. — С. 1177—1206.
6. *Wang S., Teng Y., Perdikaris P.* Understanding and Mitigating Gradient Flow Pathologies in Physics-Informed Neural Networks // *SIAM Journal on Scientific Computing*. — 2021. — Т. 43, № 5. — A3055—A3081. — DOI: 10.1137/20M1318043.
7. *Wang S., Yu X., Perdikaris P.* When and Why PINNs Fail to Train: A Neural Tangent Kernel Perspective // *Journal of Computational Physics*. — 2022. — Т. 449. — С. 110768. — DOI: 10.1016/j.jcp.2021.110768.